

# GUÍA RÁPIDA DE USO – SIMPLE IDE

VERSION: 1.1

CREADO POR: ANTONIO MANUEL GUIADO VALLE

# INTRODUCCIÓN E INSTALACIÓN

## INTRODUCCIÓN

En este texto aprenderemos a como usar las principales funcionalidades de Simple IDE y a como sacar el máximo rendimiento a esta aplicación.

## ¿QUÉ ES SIMPLE IDE?

Simple IDE es un entorno de desarrollo integrado diseñado para facilitar la programación en Java, especialmente orientado a estudiantes y desarrolladores que buscan una experiencia de codificación simplificada pero completa. Combina las funcionalidades esenciales del desarrollo con herramientas modernas con la asistencia de inteligencia artificial integrada, gestión de proyectos y ficheros intuitiva y un sistema de documentación integrado.

## INSTALACIÓN

Simple IDE tiene una alta compatibilidad multiplataforma. Dependiendo de donde quieras usarlo, tendras unas opciones u otras.

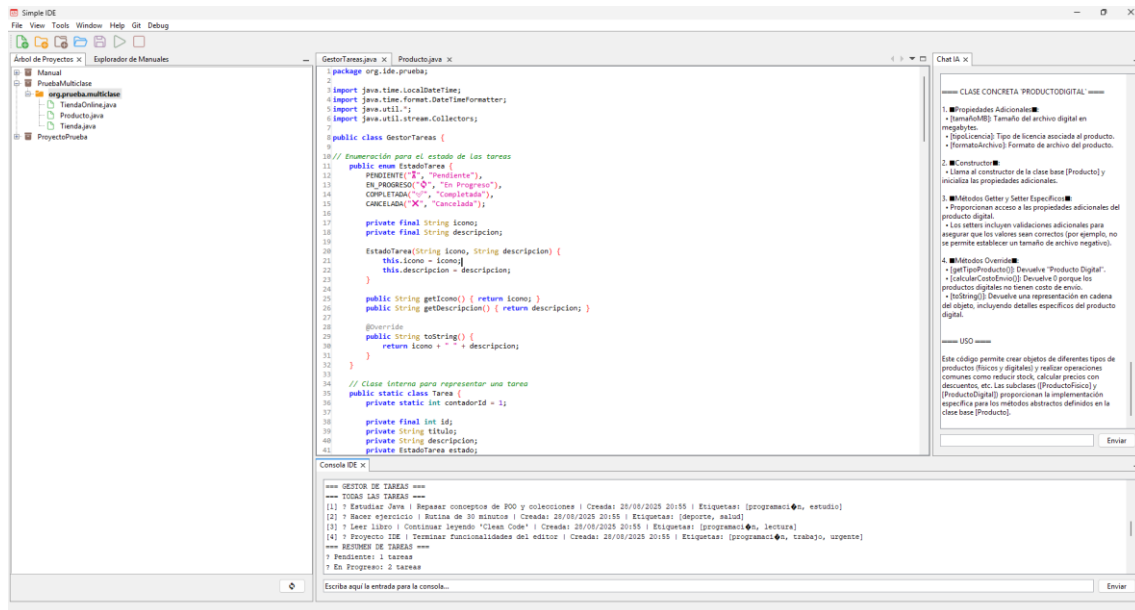
**Importante:** Excepto para la versión en navegador, se necesita tener **Java 21** instalado para poder usar la aplicación.

Ahora enumeramos las diferentes opciones:

- **Windows**
  - o Se puede instalar con el instalador “simpleide-windows.exe”.
  - o Se puede usar el ZIP portable. Dentro del ZIP, hay que dirigirse a la carpeta “simpleide/bin” y hacer doble clic en el archivo “simpleide.exe” o “simpleide64.exe” (dependiendo de tu sistema si es de 32 bits o 64 respectivamente).
- **Linux**
  - o Se puede instalar con el instalador “simpleide-linux.sh”. *No se necesita Java instalado para usarlo.*
  - o Se puede usar el ZIP portable. Dentro del ZIP, hay que dirigirse a la carpeta “simpleide/bin” y hacer doble clic en el archivo “simpleide” (el archivo que no tiene extensión).
- **MacOS**
  - o Se puede usar con el .app portable. Dentro del archivo .app, vamos a la carpeta “Contents/MacOS” y ahí encontraremos el ejecutable “simpleide”.
- **Navegador / Acceso remoto**
  - o A través de la web [ide.ide-ia-chat.lol](https://ide.ide-ia-chat.lol) podremos acceder a un servicio levantado con MiniKube y noVNC poder probar nuestro IDE sin preocupaciones ni instalaciones.
  - o La experiencia con este servicio puede verse afectada por su conexión a internet, las personas conectadas y el horario del servidor.
  - o Este servicio estará abierto de 8 AM a 8 PM de lunes a viernes hasta mediados de octubre. Más información a lo largo del documento.

## INTERFAZ PRINCIPAL

### VISTA GENERAL



Aquí tenemos un primer vistazo a Simple IDE. Tiene un estilo simple y sencillo que ayudará a hacerse con él de manera fácil y rápida. Además, es modular para que cada componente este separado del resto de una manera clara. Estos componentes o módulos también puedes configurarse como se quiera, cambiando su tamaño, posición, abriéndolo como una ventana emergente o quitándolos si no nos sirve. Y todo esto simplemente usando nuestro ratón.

### DESCRIPCIÓN DE LOS COMPONENTES

#### ÁRBOL DE PROYECTOS

El componente que tenemos a la izquierda muestra la estructura de archivos de tus diferentes proyectos y carpetas. Permite ver los diferentes paquetes y clases, además de poder organizar, renombrar o eliminar los archivos que queramos. Al hacer doble clic en uno de los archivos, estos se abren directamente en el editor de código (componente central).

Si nos fijamos, veremos que este componente tiene una pestaña llamada explorador de manuales, lo veremos más adelante.

#### EDITOR DE CÓDIGO

El componente central es donde escribes el código Java. Incluye numeración de líneas, resaltado de sintaxis con colores (palabras reservadas en púrpura, clases en amarillo, strings en naranja, etc.), y autocompletado básico. Soporta pestañas múltiples para trabajar con varios archivos simultáneamente.

#### CONSOLA

El componente inferior funciona como terminal integrada. Muestra los resultados de compilación, errores de código, y la salida de tus programas cuando los ejecutas. También permite entrada de datos del usuario mediante teclado cuando el programa lo requiera (Scanner, etc.).

---

## CHAT CON IA (QWEN 2.5)

El componente lateral derecho es el que integra el asistente Qwen 2.5 especializado en programación. Permite hacer preguntas sobre código, solicitar explicaciones de errores, y recibir sugerencias de mejoras. Las respuestas incluyen formato con títulos, listas y código resaltado para mayor legibilidad. El historial de conversación se mantiene durante la sesión actual.

---

## EXPLORADOR DE MANUALES

Por último, acompañando al árbol de proyectos, tenemos el explorador de manuales. Es una pequeña sección con manuales útiles sobre Java, Git o sobre Simple IDE. Muy útil para consultar información sin salir del IDE en ningún momento.

## FUNCIONES BÁSICAS

### BARRA DE HERRAMIENTAS

Situada encima de los componentes tenemos la barra de herramientas. Esta incluye siete iconos diferentes, cada uno responsable de una función diferente.



---

## NUEVO PROYECTO

Empezamos con el icono marrón, el tercero empezando por la izquierda. Este botón permite crear la estructura base de un proyecto Java, estableciendo los directorios principales y los archivos de configuración necesarios. Al crear un proyecto nuevo, se genera automáticamente la jerarquía de carpetas de Java. Esta creación se lleva a cabo mediante un pequeño ayudante o wizard que nos ayudará con los diferentes pasos.

---

## NUEVO PAQUETE

El icono amarillo, a la izquierda del botón “Nuevo proyecto”, crea paquetes para los proyectos que tengamos abiertos en el mismo momento. Estos paquetes permiten la mejor organización de nuestro proyecto y nuestro código. Es obligatorio su uso para crear un nuevo archivo. Al igual que antes, también lo acompaña un pequeño ayudante para que crear la carpeta sea sencillo.

---

## NUEVO ARCHIVO

Ahora vamos con el icono verde, el primero empezando por la izquierda. Con este botón crearemos archivos de código fuente Java con la estructura básica de clase. El archivo se genera con el template correspondiente al tipo de clase seleccionado y se posiciona automáticamente en el paquete designado. Además, se incluyen otros template para agilizar la programación al igual que, mediante el gestor de templates, se pueden añadir más.

---

## ABRIR PROYECTO

El icono azul del medio es el encargado de abrir proyectos previamente creados, que mostraran sus paquetes y archivos en el árbol de proyectos. Además, así se podrán crear ficheros y ejecutar el código de este.

---

## GUARDAR

El icono de guardar, el morado, no está activo hasta que se cambie la información o el código de uno de nuestros ficheros mediante el editor de código. Diferenciamos cuando los botones están activos o no con el color de los iconos. Si aparecen más claros y fuertes, están activos, si aparecen más apagados, están desactivados. Además, al guardar, se ejecuta el compilador incremental, ahorrándonos tiempo y viendo posibles errores.

---

## EJECUTAR

El otro botón verde, el que tiene forma de flecha, es el encargado de ejecutar nuestro código. Este botón solo está activo cuando se tiene el foco en una de las clases Java que tengamos abiertas en el editor de código (tener el foco significa que el ultimo clic o tabulación haya acabado en el componente de editor de código). Al pulsar el botón, ejecutamos el proyecto de la clase en la que estemos, en un proceso separado, permitiendo la interacción con la aplicación mientras se mantiene el IDE funcional. La salida del programa se muestra en la consola de ejecución. Además, si hace falta compilar algún archivo, se compila antes de ejecutar el código.

---

## PARAR PROCESO

Por último, tenemos el botón rojo con forma cuadrada. Este botón detiene inmediatamente cualquier proceso de ejecución activo, liberando los recursos del sistema y devolviendo el control completo al IDE. Esta función es especialmente útil para programas con bucles infinitos o procesos que requieren terminación manual. Además, este botón solo está activo si se hay un proceso de ejecución activo.

## FLUJO DE TRABAJO RECOMENDADO

Para ir haciéndose a la forma de trabajar en Java, recomiendo el siguiente flujo de trabajo para conseguir una buena experiencia usando Simple IDE:

1. **Crear proyecto:** Así establecemos la base del desarrollo.
2. **Crear paquetes:** Con estos paquetes organizaremos la estructura del código.
3. **Crear ficheros:** Implementamos los ficheros y clases necesarias.
4. **Abrir ficheros y editarlos:** Probar las diferentes funcionalidades.
5. **Guardar y compilar:** Guardamos los cambios y verificamos que todo vaya bien.
6. **Ejecutar:** Probar la funcionalidad del código.
7. **Parar:** Si fuera necesario, se clicca en parar.

Este sería el ciclo básico. Además, sería conveniente probar el chat con Qwen y explorar las demás funcionalidades de la aplicación.

## CÓDIGO DE EJEMPLO

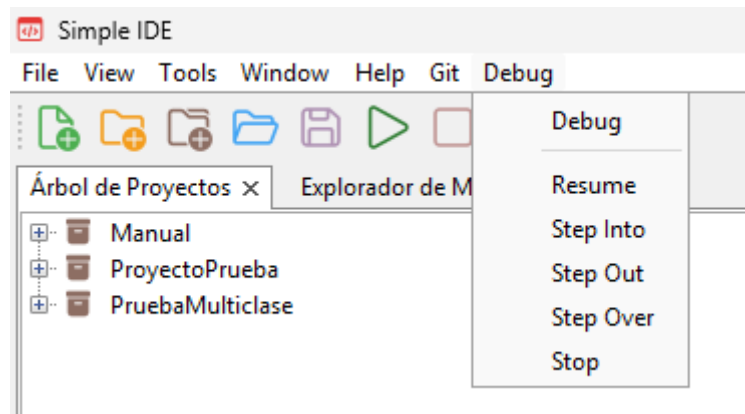
Para ayudar a hacerse con el nuevo entorno, se ha incluido en la descarga la carpeta “Código de ejemplo” con tres proyectos listos para usar. Estos ejemplos se pueden abrir con el botón de “Abrir proyecto” y cogiendo la carpeta del ejemplo deseada.

Estos ejemplos oscilan en complejidad y dificultad, así que no dudes en mirarlos detenidamente, ejecutarlos y modificarlos a tu antojo. Si tienes cualquier duda de cómo funciona el código, siempre le podrás preguntar a Qwen.

## FUNCIONES AVANZADAS

Las siguientes funciones son para personas que ya estén acostumbradas al flujo de trabajo en Java y con Simple IDE. Ahora presentamos las diferentes funciones avanzadas, como la ayuda a la depuración de código y los comandos Git.

### DEPURACIÓN DE CÓDIGO



Ahora explicaremos las diferentes herramientas que ofrece la aplicación para la ayuda en la depuración del código. Como vemos, la mayoría de las acciones se encuentran en el menú “Debug” en la barra de herramientas.

---

### BREAKPOINTS

Para la creación de breakpoints en cualquier clase Java, lo que necesitamos hacer es clicar en el número de la línea en la que queremos poner el breakpoint. Haciendo esto, se nos abrirá una pequeña columna que nos marcará con un círculo rojo la línea donde hemos puesto el breakpoint. Queda de la siguiente manera:

```
Producto.java x GestorTareas.java x
1 package org.ide.prueba;
2
3 import java.time.LocalDateTime;
4 import java.time.format.DateTimeFormatter;
5 import java.util.*;
6 import java.util.stream.Collectors;
7
8 public class GestorTareas {
9
10 // Enumeración para el estado de las tareas
11 public enum EstadoTarea {
12     PENDIENTE("P", "Pendiente"),
13     EN_PROGRESO("Q", "En Progreso"),
14     COMPLETADA("X", "Completada"),
15     CANCELADA("X", "Cancelada");
16
17     private final String icono;
18     private final String descripcion;
19
20     EstadoTarea(String icono, String descripcion) {
21         this.icono = icono;
22         this.descripcion = descripcion;
23     }
24
25     public String getIcono() { return icono; }
26     public String getDescripcion() { return descripcion; }
27
28     @Override
29     public String toString() {
30         return icono + " " + descripcion;
31     }
32 }
33
34 // Clase interna para representar una tarea
35 public static class Tarea {
36     private static int contadorId = 1;
37     private final int id;
38 }
```

Para quitar los breakpoints, hacemos lo mismo que para ponerlos, doble clic en el número de la línea del código. Hay que avisar también que, por legibilidad, en este estado se desactiva la ayuda a la sintaxis. Para volver a activarla basta con clicar en la nueva columna con el botón derecho y esta se cerrará y el analizador volverá a funcionar.

## DEBUG

Al clicar en debug en la barra de herramientas, empezara el modo depuración de código. En este modo veremos marcado con una flecha verde la línea de código en la que se ha parado el código. Esta flecha sustituirá al icono de los breakpoints cuando coincidan. También se abren otros dos útiles componentes, la pila de llamadas, que almacena las diferentes llamadas realizadas en el código, y las variables locales, que en todo momento te dicen el valor de las variables del código.

```
Producto.java x GestorTareas.java x
175
176 // Método main para probar el gestor
177 public static void main(String[] args) {
178     System.out.println("=== GESTOR DE TAREAS ===\n");
179
180     GestorTareas gestor = new GestorTareas();
181
182     // Crear tareas de ejemplo
183     Tarea tarea1 = gestor.crearTarea("Estudiar Java", "Repasa
184     tarea1.agregarEtiqueta("estudio");
185     tarea1.agregarEtiqueta("programación");
186
187     Tarea tarea2 = gestor.crearTarea("Hacer ejercicio", "Ruti
188     tarea2.agregarEtiqueta("salud");
189     tarea2.agregarEtiqueta("deporte");
190
191     Tarea tarea3 = gestor.crearTarea("Leer libro", "Continuar
192     tarea3.agregarEtiqueta("lectura");
193     tarea3.agregarEtiqueta("programación");
194
195     Tarea tarea4 = gestor.crearTarea("Proyecto IDE", "Termina
196     tarea4.agregarEtiqueta("trabajo");
197     tarea4.agregarEtiqueta("programación");
198     tarea4.agregarEtiqueta("urgente");
199
200     // Cambiar estados de algunas tareas
201     gestor.cambiarEstado(1, EstadoTarea.COMPLETADA);
202     gestor.cambiarEstado(2, EstadoTarea.EN_PROGRESO);
203     gestor.cambiarEstado(4, EstadoTarea.EN_PROGRESO);
204
205     // Mostrar información
206     gestor.mostrarTodasLasTareas();
207     gestor.mostrarResumen();
208
209     // Usar streams para filtros
210     System.out.println("\n=== TAREAS DE PROGRAMACIÓN ===");
211     gestor.buscarTareasPorEtiqueta("programación")
212         .forEach(System.out::println);
213 }
```

**Pila de llamadas**

```
#0 org.ide.prueba.GestorTareas.main (línea 195)
```

**Variables locales**

| Variable | Valor                      |
|----------|----------------------------|
| gestor   | instance of org.ide.pru... |
| tarea1   | instance of org.ide.pru... |
| tarea2   | instance of org.ide.pru... |
| tarea3   | instance of org.ide.pru... |

Una vez empezados con la depuración, entramos en detalle de las demás opciones.

---

## RESUME

Esta acción reanuda la ejecución normal del programa después de haber sido pausado por un breakpoint o por una acción de debug manual. El programa continuará ejecutándose hasta encontrar el siguiente breakpoint, finalizar normalmente, o hasta que se detenga manualmente.

---

## STEP INTO

Clicando en esta opción avanzamos la ejecución una línea, pero si la línea actual contiene una llamada a un método, el debugger entra dentro de ese método para permitir el debug paso a paso del código interno. Es la opción más detallada de seguimiento de ejecución.

---

## STEP OUT

Continúa la ejecución del método actual hasta su finalización y pausa inmediatamente después de retornar al método padre. Útil cuando se ha entrado en un método con Step Into pero se desea salir rápidamente para continuar con el flujo principal.

---

## STEP OVER

Con esta opción se avanza la ejecución una línea, pero si la línea contiene llamadas a métodos, las ejecuta completamente sin entrar en ellas. El debugger se detiene en la siguiente línea del método actual, manteniendo el mismo nivel de contexto.

---

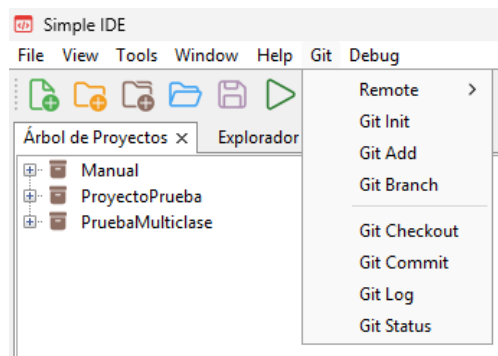
## STOP

Finaliza inmediatamente tanto la ejecución del programa como la sesión de debug activa. Todos los breakpoints permanecen configurados, pero el proceso actual se termina completamente y se liberan todos los recursos asociados. También se cierran los componentes de la pila de llamadas y las variables locales.

## GIT

Git es una herramienta para el control de versiones, facilitando el seguimiento de cambios en el código a lo largo del tiempo y la colaboración entre desarrolladores. En el IDE hemos añadido algunos de sus comandos más utilizados para no tener que salir del IDE ni entrar en consola para hacer las diferentes acciones. Estas acciones las encontramos en la sección Git de la barra de herramientas.

Si no se entiende bien con las siguientes explicaciones cómo funciona Git, en el explorador de manuales se incluye una guía de Git extensa para quien quiera sacar el máximo provecho.





---

## GIT INIT

Crea un nuevo repositorio Git local en el directorio del proyecto, estableciendo la estructura base para el control de versiones. Equivale a ejecutar “*git init*” en consola.

---

## GIT ADD

Agrega los archivos que hayan sido añadidos, cambiados o editados al siguiente commit. Equivale a “*git add .*” en consola

---

## GIT BRANCH

Permite crear, listar, o eliminar ramas en el repositorio. Facilita el trabajo con diferentes líneas de desarrollo paralelas. Viene acompañado de un pequeño selector de acciones. Equivale a “*git branch*” en consola.

---

## GIT CHECKOUT

Permite moverse entre diferentes ramas del proyecto o restaurar archivos a versiones anteriores. Actualiza el workspace con el contenido de la rama seleccionada. También incluye un selector de acciones para facilitar su uso. Equivale a “*git checkout*” en consola.

---

## GIT COMMIT

Crea un punto de guardado permanente con los cambios que están en el área de staging, incluyendo un mensaje descriptivo. Equivale a “*git commit*” en consola.

---

## GIT LOG

Muestra la lista de commits realizados en la rama actual, incluyendo información como autor, fecha, mensaje y hash de cada commit. Equivale a “*git log*” en consola.

---

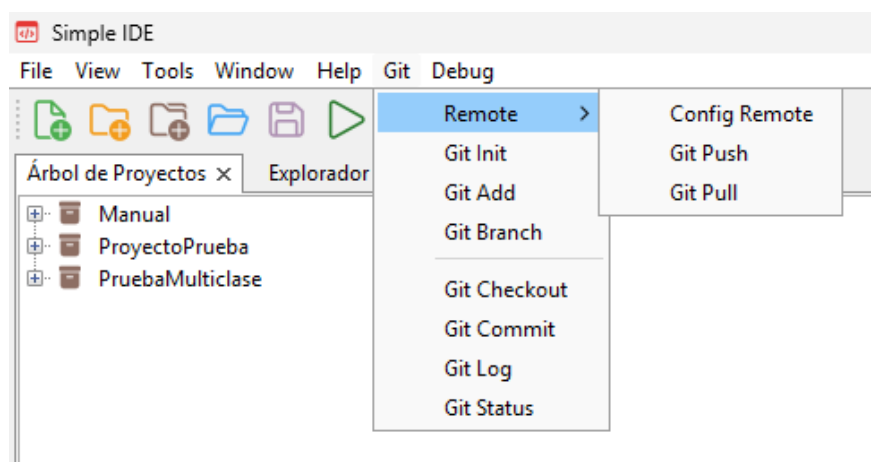
## GIT STATUS

Proporciona información sobre archivos modificados, archivos en staging, y el estado general del working directory comparado con el último commit. Equivale a “*git status*” en consola.

---

## REMOTE

Ahora vemos las opciones que tenemos para trabajar con repositorios remotos.



---

## CONFIG REMOTE

Permite añadir, modificar o eliminar las conexiones con repositorios remotos (como GitHub, GitLab, etc.). Establece las URLs para operaciones de push y pull. Incluye un pequeño ayudante para indicar que queremos hacer. Equivale a “*git remote*” en consola.

---

## GIT PUSH

Sube los commits realizados localmente al repositorio remoto configurado, sincronizando los cambios con el servidor. Equivale a “*git push*” en consola.

---

## GIT PULL

Obtiene los últimos cambios del repositorio remoto y los integra automáticamente en la rama local actual. Equivale a “*git pull*” en consola.

Como podemos observar, tenemos una gran variedad de opciones para controlar nuestro repositorio, tanto local como remoto. Aun así, para cualquier otra función o acción más personalizada, siempre se recomienda usar Git desde consola, pero Simple IDE ofrece la comodidad de usar gran parte de su funcionalidad sin salir del IDE.

## USO DE SIMPLE IDE DESDE NAVEGADOR

Existe la posibilidad de usar Simple IDE con simplemente acceso a internet y tu navegador de confianza. Para ello, simplemente hay que ir a la url <https://ide.ide-ia-chat.lol/> para empezar a usar la aplicación. Aun así, hay ciertas limitaciones que tener en cuenta:

- Ahora mismo solo hay capacidad para que lo use una persona a la vez. Si ves la pantalla de noVNC pero no puedes entrar, significa que hay otra persona usandolo, vuelve a intentarlo más tarde.
- Como mencionamos antes, este servicio estará abierto de 8 AM a 8 PM de lunes a viernes hasta mediados de octubre.
- Al ser un servicio compartido, los proyectos se comparten y acciones se comparten y se reinician cada día.

Particularmente recomendamos usar esta opción para probar de manera rápida lo que ofrece Simple IDE, pero, si se quiere una experiencia mejor, se recomienda la instalación o uso local del programa.

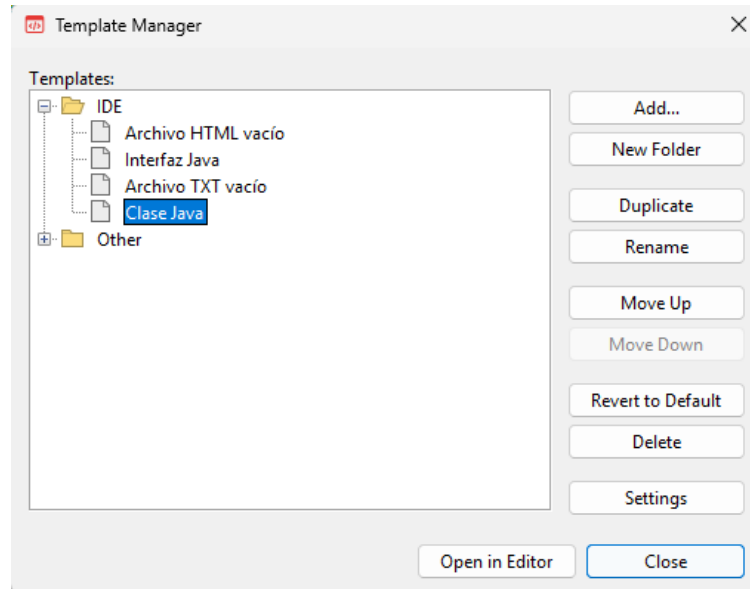
## USO DEL CHAT IA CON QWEN

Esté chat estará disponible las mismas horas y días que está disponible el programa a través del navegador. No hay límites de uso, pero si me gustaría dar varios consejos:

- Al ser un servidor personal, la potencia y rapidez de respuesta no es tan potente como otros sistemas de la competencia, pero esto es totalmente gratis.
- Por la misma razón, intentar no abusar del chat. Esto puede crear fallos en el servidor y cuelgues.
- Puedes usar el chat de muchas maneras diferentes, ya sea para preguntar dudas sobre programación, enviar código para que te lo corrija o mejore, pedirle que te cree cierta función y un sinfín más de opciones.
- Aunque este IDE está totalmente enfocado a Java, eso no impide que puedas preguntarle por otros lenguajes a Qwen.

## TEMPLATE MANAGER

Ahora toca describir nuestro template manager o gestor de plantillas. Para acceder al gestor, basta con ir a la sección “Tools” en la barra de herramientas. Cuando se abran las opciones, veremos la opción “Templates” y ahí es donde se veremos el gestor.



Como podemos observar, tenemos diferentes opciones. Las más importantes son:

- **Add:** Para añadir cualquier archivo compatible de nuestro PC como plantilla.
- **Rename:** Para renombrar una plantilla ya existente.
- **Delete:** Para borrarla una plantilla ya existente.
- **Open in editor:** Para editar dentro del IDE el archivo como queramos. (Si editamos una plantilla, para guardar los cambios cerramos la pestaña abierta y ahí tendremos la opción para guardar).
- **Revert to default:** Para revertir cambios en una plantilla modificada.

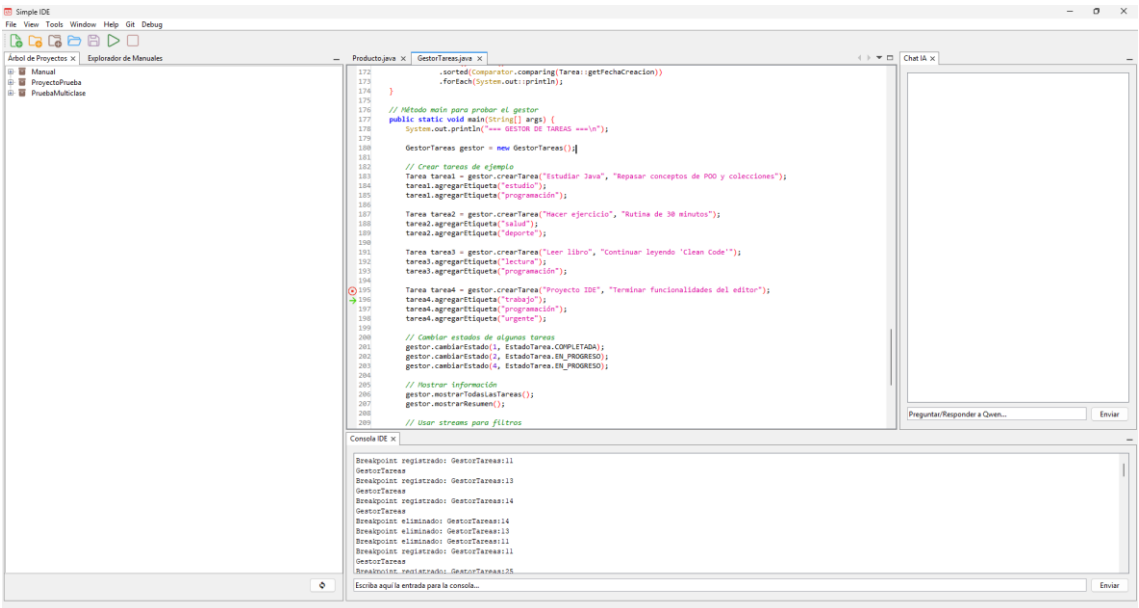
**IMPORTANTE:** Solo las plantillas que estén en la carpeta **IDE** son las que se utilizarán con el ayudante de creación de nuevos ficheros.

## CONFIGURACIÓN Y PERSONALIZACIÓN

Además de todas las funciones que hemos descrito, también podemos personalizar el IDE. En particular destacamos la posibilidad de cambios de tema, la posibilidad de mover los componentes como se quiera y organizarlos a tu gusto, llegando a poder usarlos como ventanas emergentes en varias pantallas a la vez.

Para cambiar el tema, hay que entrar en la sección “Tools” de la barra de herramientas, clicar en “Options”, luego “Appearance” hasta llegar a la sección “Look And Feel”, donde podremos seleccionar el tema que más nos guste.

# TEMA CLARO



# TEMA OSCURO

